

ISR Drone Feed Integration

Offline Common Operating Picture (COP)

System Architecture, Software Design, Integration and Testing Document

Project Technical Documentation

September 21, 2026

Project Type: Offline ISR / Geospatial Visualization / Sensor Integration

Primary Objective: Integrate drone-derived sensor information into an offline geospatial software environment

Operating Constraint: Fully functional without Internet connectivity

Contents

1	Executive Summary	4
2	Project Objectives	5
3	Scope	5
3.1	Software Scope	5
3.2	Potential Platform Scope	6
4	High-Level System Architecture	6
5	Recommended Technology Stack	7
5.1	Application Framework	7
5.2	Mapping	8
5.3	Database	8
5.4	Video	8
5.5	Coordinate Engine	9

6	Core Data Flow	9
6.1	Sensor Stream	9
6.2	Telemetry Stream	10
7	Drone Integration Architecture	10
7.1	Common Telemetry Model	10
7.2	Platform Adapter	11
8	Video Feed Architecture	12
8.1	Development Stage	12
8.2	Hardware Stage	12
9	Offline Architecture	13
10	Map System	14
11	Coordinate and Grid Reference System	15
11.1	MGRS	15
11.2	Indian Grid	15
12	Telemetry Engine	15
13	Telemetry Simulator	16
14	Video and Telemetry Synchronization	17
15	Georeferencing Architecture	18
16	Observation / Marker System	19
17	Local Database	20
17.1	Missions	20
17.2	Platforms	20
17.3	Telemetry	20
17.4	Observations	21
18	Mission Recording	21
19	Mission Replay	22
20	User Interface Architecture	23
21	Recommended Software Structure	23

22 API / Internal Module Interfaces	25
22.1 Coordinate Engine	25
22.2 Telemetry Engine	25
22.3 Mission Engine	26
22.4 Observation Engine	26
23 Development Strategy	26
23.1 Phase 1 – User Interface Prototype	26
23.2 Phase 2 – Offline Mapping	27
23.3 Phase 3 – Coordinate Engine	27
23.4 Phase 4 – Telemetry Simulator	27
23.5 Phase 5 – Recorded Video	27
23.6 Phase 6 – Georeferenced Observation	28
23.7 Phase 7 – Hardware Adapter	28
24 Testing Strategy	29
24.1 Coordinate Testing	29
24.2 Map Testing	29
24.3 Telemetry Testing	30
24.4 Video Testing	30
24.5 Observation Testing	31
24.6 Offline Testing	31
24.7 Failure Testing	32
25 Test Matrix	32
26 Data Integrity and Auditability	33
27 Security and Offline Reliability	34
28 Performance Requirements	34
29 Hardware Integration Checklist	35
30 Important Engineering Constraint	35
31 Recommended MVP	36
32 Final Architecture	37
33 Development Philosophy	37
34 Conclusion	38

1 Executive Summary

The objective of this project is to develop an offline software system capable of receiving or replaying an ISR drone sensor feed, processing associated telemetry, and visualizing the resulting information on a digital map.

The software is intended to function as a Common Operating Picture (COP) style application. The system combines:

- Drone telemetry
- Sensor/video information
- Geospatial coordinates
- Grid references
- Platform position and track
- Sensor orientation
- Map layers
- Operator observations
- Mission/event logging
- Offline data storage
- Replay and testing capabilities

The system must not depend on an Internet connection during normal operation. The overall concept is:

**DRONE / RECORDED SENSOR DATA → DATA INGESTION →
NORMALIZATION → GEOREFERENCING → MAP + VIDEO →
OPERATOR OBSERVATION**

The system should be designed so that the user interface is independent of the specific drone manufacturer.

This is important because different platforms may provide different telemetry formats, communication interfaces, sensor payloads, and video transport mechanisms.

Therefore, the software will use a common internal data model and platform-specific adapters.

2 Project Objectives

The primary objectives are:

1. Develop an offline geospatial visualization application.
2. Display an offline map without Internet access.
3. Display drone/platform position.
4. Display historical platform tracks.
5. Display available telemetry.
6. Display recorded or connected sensor/video feeds.
7. Synchronize video and telemetry using timestamps.
8. Support grid-reference display.
9. Support geospatial observation marking.
10. Store mission and observation information locally.
11. Provide mission replay capability.
12. Provide structured data export.
13. Support simulator-based testing before hardware integration.
14. Provide a modular architecture for multiple drone platforms.

3 Scope

The project is divided into several major areas.

3.1 Software Scope

The software consists of:

- Desktop application
- Map engine
- Coordinate/grid engine
- Telemetry engine

- Video ingestion/playback engine
- Georeferencing engine
- Observation/marker engine
- Local database
- Mission recorder
- Mission replay engine
- Simulator
- Export system
- Testing framework

3.2 Potential Platform Scope

The integration architecture should be capable of supporting platforms such as:

- Asteria A200
- ideaForge Q6
- ULPGM / platforms supplied by Raphe mPhibr
- FPV platforms
- Other platforms providing compatible telemetry/video interfaces

The actual integration protocol for each platform must be determined from the official technical/interface documentation supplied for that platform.

The application must therefore not assume that all platforms use the same protocol.

4 High-Level System Architecture

The system follows a layered architecture.

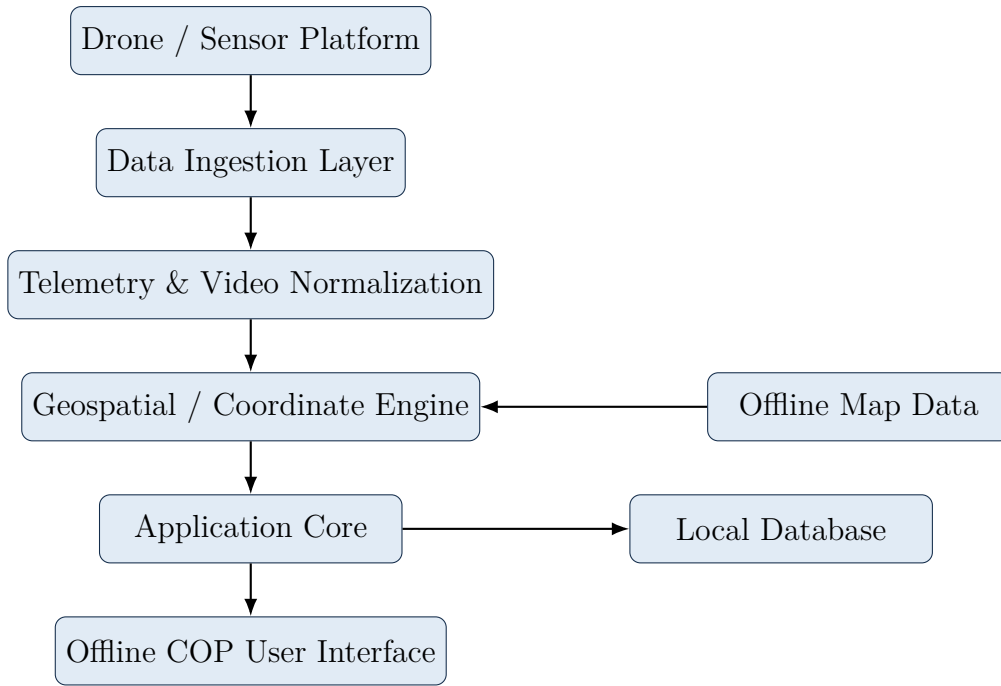


Figure 1: High-Level System Architecture

The architecture separates hardware-specific communication from the rest of the application.

This allows a new drone platform to be integrated by creating a new adapter rather than rewriting the entire application.

5 Recommended Technology Stack

A desktop-first architecture is recommended because the application must operate offline and may need direct access to local files, video streams, serial/network interfaces, and local databases.

5.1 Application Framework

A recommended stack is:

- Electron
- React
- JavaScript
- Node.js

Electron provides a desktop application container while React provides the user interface.

Node.js can provide local application services and communication with system resources.

5.2 Mapping

The mapping layer should support locally stored map data.

Possible formats include:

- MBTiles
- Raster tiles
- Vector tiles
- GeoJSON
- GeoPackage

No Internet map server should be required during normal operation.

5.3 Database

SQLite is recommended for the initial offline application.

Advantages include:

- No external database server
- Single local database file
- Fully offline
- Easy backup
- Easy deployment
- Suitable for mission/event records

5.4 Video

The system should initially support recorded video.

This allows development and testing without depending on physical hardware.

Potential video sources include:

- MP4 recordings
- Local video files
- Test streams
- Hardware-provided streams once the interface is known

5.5 Coordinate Engine

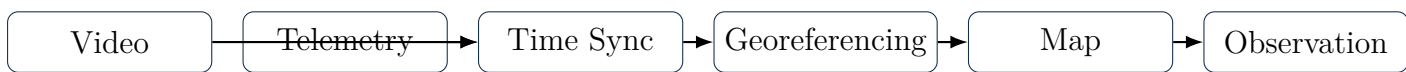
The coordinate engine should provide a common abstraction for:

- Latitude/Longitude
- WGS84
- UTM
- MGRS
- Required Indian grid/reference system

The exact Indian grid/reference specification must be confirmed with the project requirements before implementation.

6 Core Data Flow

The fundamental data flow is:



The system therefore has two major data streams:

6.1 Sensor Stream

Sensor

|

v

Video / Image Feed

|

v

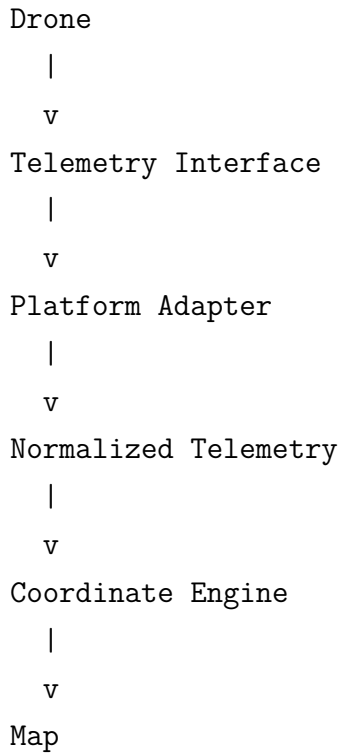
Video Decoder

|

v

Application Video Panel

6.2 Telemetry Stream



The two streams are synchronized through timestamps.

7 Drone Integration Architecture

The application should not directly couple the user interface to a specific drone.

Instead, the architecture should use a platform adapter.

7.1 Common Telemetry Model

All platform-specific telemetry should be converted into a common internal representation.

Example:

```
{
  timestamp: 1720000000000,

  platformId: "UAV-01",

  position: {
    latitude: 19.0760,
    longitude: 72.8777,
    altitudeMSL: 450.0
  },
}
```

```

    attitude: {
        roll: 0.0,
        pitch: 2.0,
        yaw: 145.0
    },

    velocity: {
        groundSpeed: 12.4
    },

    sensor: {
        azimuth: 150.0,
        elevation: -35.0,
        hfov: 45.0,
        vfov: 30.0
    }
}

```

The actual field names and units should be finalized during implementation.

7.2 Platform Adapter

The adapter converts manufacturer-specific information into the common model.

Platform Feed

|

v

Platform Adapter

|

v

Common Telemetry Model

|

+-----> Map

|

+-----> Georeferencing

|

+-----> Mission Recorder

This architecture allows:

Asteria Adapter

ideaForge Adapter
 Raphe Adapter
 FPV Adapter
 Test Simulator Adapter

to all produce the same internal telemetry structure.

8 Video Feed Architecture

Video should be treated as an independent subsystem.

8.1 Development Stage

The first implementation should use recorded video.

Example:

```
test_video.mp4
  |
  v
Video Player
  |
  v
COP Sensor Window
```

This makes it possible to develop the user interface and testing framework without the drone.

8.2 Hardware Stage

Once the actual drone interface is documented:

```
Drone Sensor
  |
  v
Supported Transport
  |
  v
Video Receiver
  |
  v
Decoder
```

|
v
COP Application

The exact protocol must be obtained from the relevant platform documentation.

The application should not assume RTSP, UDP, HDMI capture, or another transport until the actual platform interface is confirmed.

9 Offline Architecture

Offline operation is a fundamental requirement.

The system should remain functional when:

- Wi-Fi is disabled
- Ethernet is disconnected
- Internet access is unavailable
- External map services cannot be reached

The following resources should be stored locally:

- Map data
- Coordinate libraries
- Application configuration
- Mission database
- Test videos
- Telemetry recordings
- Mission logs

The application should never depend on:

- Google Maps API
- Online map tiles
- Cloud database
- Cloud authentication
- Remote geocoding
- Internet-based coordinate conversion

during normal offline operation.

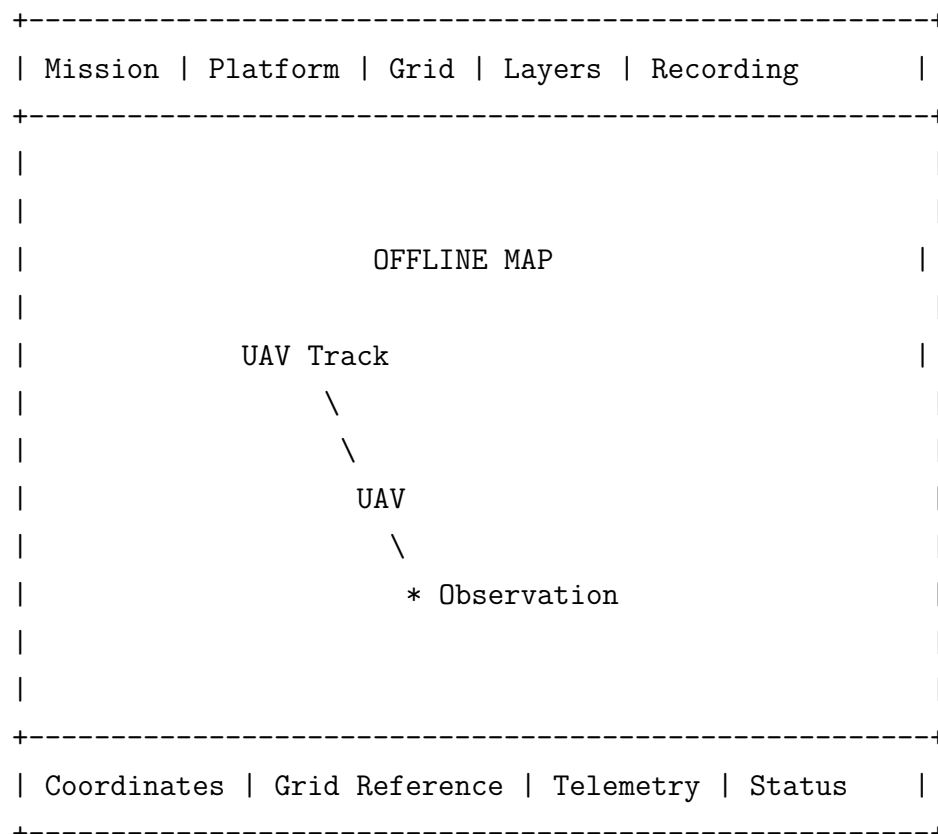
10 Map System

The map is the central visualization component.

It should provide:

- Pan
- Zoom
- Position display
- Platform marker
- Platform track
- Grid overlay
- Cursor coordinate
- Observation markers
- Sensor footprint where supported
- North indicator
- Scale information

A conceptual layout is:



11 Coordinate and Grid Reference System

The coordinate engine is one of the most important components.

The internal application should preferably maintain a high-precision geographic representation and convert it to the display grid when required.

The conceptual relationship is:

Latitude/Longitude $\leftrightarrow$ Projected Coordinate $\leftrightarrow$ Grid Reference

The application should support a configurable coordinate reference system.

11.1 MGRS

If MGRS is required, the system should convert geographic coordinates to MGRS.

For example:

```

Latitude
Longitude
    |
    v
WGS84
    |
    v
UTM Zone
    |
    v
MGRS

```

The implementation must be tested against known reference points.

11.2 Indian Grid

If the operational requirement specifies an Indian grid/reference system, the exact datum, projection, zone, and transformation parameters must be confirmed before implementation.

The application should not simply label WGS84 coordinates as an Indian Grid reference.

12 Telemetry Engine

The telemetry engine receives normalized platform information.

Typical information may include:

- Timestamp
- Platform identifier
- Latitude
- Longitude
- Altitude
- Heading
- Roll
- Pitch
- Speed
- Sensor orientation
- Sensor field of view

The telemetry engine is responsible for:

1. Receiving telemetry
2. Validating telemetry
3. Normalizing units
4. Timestamping
5. Updating platform state
6. Sending state to the map
7. Recording telemetry

13 Telemetry Simulator

Before connecting real hardware, a simulator should be developed.

The simulator can generate:

- Platform position
- Heading
- Altitude

- Speed
- Sensor orientation
- Timestamp

Example:

```
{
  "platformId": "SIM-UAV-01",
  "latitude": 19.0760,
  "longitude": 72.8777,
  "altitude": 300,
  "heading": 120,
  "speed": 10,
  "timestamp": 1720000000000
}
```

The simulator should support predetermined tracks.

For example:

Point A

|

v

Point B

|

v

Point C

|

v

Point D

This allows repeatable testing.

14 Video and Telemetry Synchronization

Synchronization is necessary if an observation is associated with a specific video frame.

Each telemetry record and video frame should have a time reference.

Conceptually:

$$T_{video} \approx T_{telemetry}$$

For a selected video frame:

T_f = frame timestamp

The application finds the closest telemetry record:

$$T_i = \arg \min_i |T_i - T_f|$$

The corresponding platform state can then be associated with that frame.

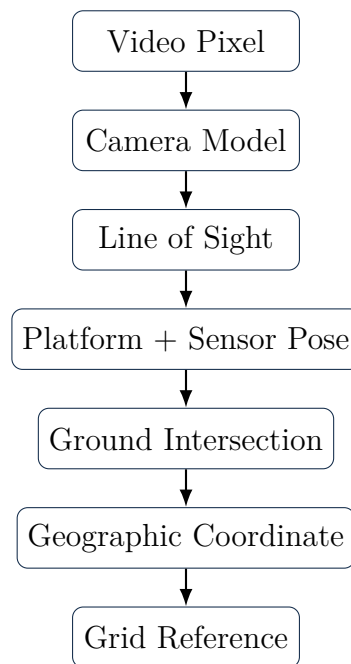
This enables the system to answer:

“What was the platform and sensor state at the moment this frame was captured?”

15 Georeferencing Architecture

Georeferencing converts sensor observations into geographic positions.

The conceptual pipeline is:



The calculation depends on information such as:

- Platform position
- Platform altitude
- Platform attitude
- Sensor orientation

- Camera calibration
- Field of view
- Image dimensions
- Ground/terrain model

For an initial prototype, a simplified ground model may be used.

A more accurate implementation may use elevation/terrain data.

16 Observation / Marker System

The application should allow the operator to create a geospatial observation from the map or sensor interface.

An observation should contain structured information rather than only a visual marker.

Example:

```
{
  "id": "OBS-0001",

  "timestamp": 1720000000000,

  "source": "VIDEO",

  "platformId": "UAV-01",

  "position": {
    "latitude": 19.0760,
    "longitude": 72.8777
  },

  "gridReference": "...",

  "status": "OBSERVED",

  "operatorNote": "Observation created from recorded sensor feed"
}
```

The system should distinguish between:

1. Sensor observation
2. Calculated geographic position

3. Operator-confirmed observation

These should not automatically be treated as the same level of certainty.

17 Local Database

SQLite can provide the local persistence layer.

A possible schema is:

17.1 Missions

```
missions
-----
id
name
created_at
started_at
ended_at
status
description
```

17.2 Platforms

```
platforms
-----
id
mission_id
platform_name
platform_type
adapter_type
```

17.3 Telemetry

```
telemetry
-----
id
mission_id
platform_id
timestamp
latitude
```

longitude
altitude
roll
pitch
yaw
speed
sensor_azimuth
sensor_elevation

17.4 Observations

observations

id
mission_id
platform_id
timestamp
latitude
longitude
grid_reference
source
status
notes

18 Mission Recording

The system should record mission events.

Examples include:

- Mission started
- Platform connected
- Video started
- Telemetry received
- Observation created
- Observation edited
- Platform disconnected

- Mission ended

A mission event can be represented as:

```
{
  "timestamp": 1720000000000,
  "type": "OBSERVATION_CREATED",
  "missionId": "MISSION-001",
  "platformId": "UAV-01",
  "objectId": "OBS-001"
}
```

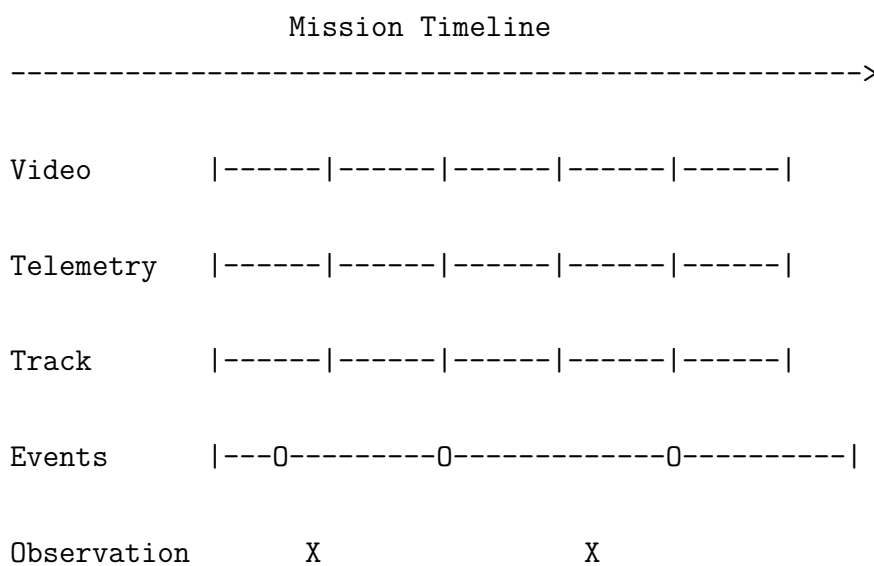
19 Mission Replay

Mission replay is strongly recommended.

A recorded mission should contain synchronized:

- Video
- Telemetry
- Platform track
- Observations
- Mission events

The replay engine reconstructs the mission timeline.



This is extremely useful for software testing because the same mission can be replayed repeatedly.

20 User Interface Architecture

The UI can be divided into:

1. Header
2. Platform panel
3. Main map
4. Sensor/video panel
5. Telemetry panel
6. Observation panel
7. Mission timeline
8. System status

Conceptual layout:

+-----+-----+-----+-----+-----+-----+					
Mission	Platforms	Layers	Grid	Record	Replay
+-----+-----+-----+-----+-----+-----+					
PLATFORM PANEL		MAP		SENSOR VIDEO	
UAV-01					
UAV-02					
+-----+-----+-----+-----+-----+-----+					
Telemetry		Observation / Event Log			
+-----+-----+-----+-----+-----+-----+					
System Status	Database	Video	Telemetry	Offline	
+-----+-----+-----+-----+-----+-----+					

21 Recommended Software Structure

A possible project structure is:

```
sentinel-cop/
|
+-- electron/
```

```
|   +-- main.js
|   +-- preload.js
|
+-- client/
|   +-- src/
|       +-- components/
|           |   +-- Map/
|           |   +-- Video/
|           |   +-- Telemetry/
|           |   +-- Platforms/
|           |   +-- Observations/
|           |   +-- Mission/
|           |
|       +-- pages/
|       +-- state/
|       +-- services/
|
+-- core/
|   +-- coordinates/
|   +-- georeferencing/
|   +-- telemetry/
|   +-- synchronization/
|   +-- observations/
|   +-- mission/
|
+-- adapters/
|   +-- asteria/
|   +-- ideaforge/
|   +-- raphe/
|   +-- fpv/
|   +-- simulator/
|
+-- database/
|   +-- migrations/
|   +-- repositories/
|
+-- maps/
|   +-- tiles/
|   +-- elevation/
|
+-- media/
```

```
|   +-- test-video/  
|  
+-- simulator/  
|   +-- telemetry/  
|   +-- missions/  
|  
+-- tests/  
|   +-- coordinates/  
|   +-- telemetry/  
|   +-- georeferencing/  
|   +-- video/  
|   +-- database/  
|   +-- offline/  
|  
+-- package.json
```

22 API / Internal Module Interfaces

Modules should communicate through well-defined interfaces.

22.1 Coordinate Engine

Example:

```
coordinateEngine.toMGRS(latitude, longitude)  
  
coordinateEngine.fromMGRS(mgrs)  
  
coordinateEngine.toIndianGrid(latitude, longitude)  
  
coordinateEngine.fromIndianGrid(grid)
```

22.2 Telemetry Engine

```
telemetryEngine.start()  
  
telemetryEngine.stop()  
  
telemetryEngine.getPlatformState(platformId)  
  
telemetryEngine.subscribe(callback)
```

22.3 Mission Engine

```
missionEngine.createMission()

missionEngine.startMission()

missionEngine.recordEvent()

missionEngine.endMission()

missionEngine.replayMission()
```

22.4 Observation Engine

```
observationEngine.createObservation()

observationEngine.updateObservation()

observationEngine.deleteObservation()

observationEngine.exportObservations()
```

23 Development Strategy

The project should be built incrementally.

23.1 Phase 1 – User Interface Prototype

Build:

- Application shell
- Map panel
- Video panel
- Telemetry panel
- Platform panel
- Observation panel

All data can initially be fake.

23.2 Phase 2 – Offline Mapping

Implement:

- Local map data
- Zoom
- Pan
- Coordinates
- Grid display

Test with the computer completely disconnected from the Internet.

23.3 Phase 3 – Coordinate Engine

Implement and test:

- WGS84
- UTM
- MGRS
- Required Indian grid/reference system

Known coordinate test cases should be created.

23.4 Phase 4 – Telemetry Simulator

Implement a simulator that generates repeatable platform tracks.

This provides a hardware-independent way of testing the map.

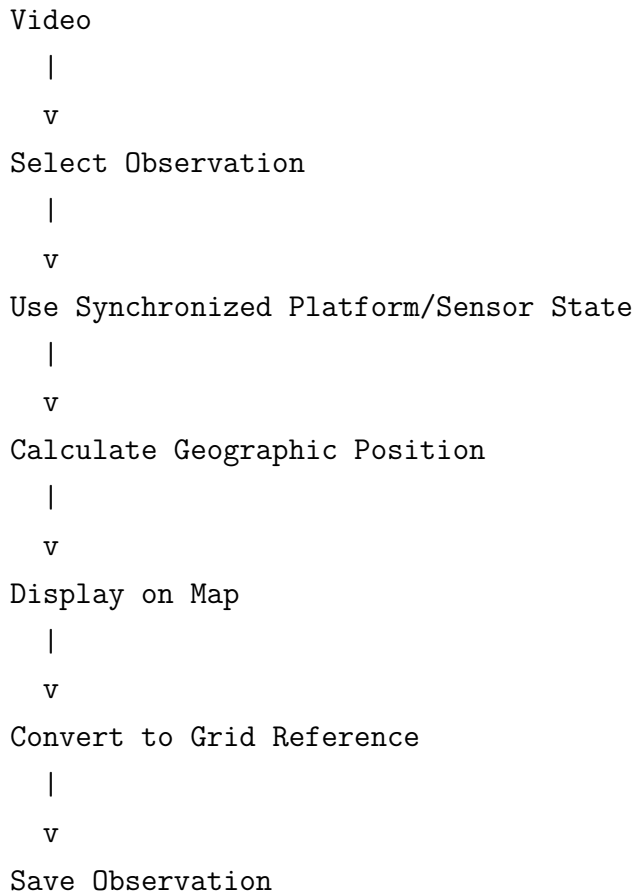
23.5 Phase 5 – Recorded Video

Add recorded sensor video.

The application should synchronize video playback with simulated telemetry.

23.6 Phase 6 – Georeferenced Observation

Implement the observation workflow:



23.7 Phase 7 – Hardware Adapter

Only after the software works with simulation and recorded data should the real platform adapter be introduced.

The hardware integration process is:

1. Obtain official interface documentation.
2. Identify telemetry protocol.
3. Identify video transport.
4. Identify available sensor metadata.
5. Implement adapter.
6. Validate incoming data.
7. Compare hardware data against known values.
8. Run controlled integration tests.

24 Testing Strategy

Testing is a major part of this project.

Testing should occur at several levels.

24.1 Coordinate Testing

Test:

- WGS84 to MGRS
- MGRS to WGS84
- WGS84 to required Indian grid
- Reverse conversions
- Boundary conditions
- Invalid coordinates
- Invalid grid references

Known reference points should be used as test vectors.

24.2 Map Testing

Test:

- Map loading
- Zoom
- Pan
- Platform display
- Track display
- Grid display
- Observation markers
- Multiple platforms
- Coordinate readout

24.3 Telemetry Testing

Test:

- Normal telemetry
- Missing telemetry
- Delayed telemetry
- Duplicate telemetry
- Invalid values
- Timestamp errors
- Platform disconnect
- Reconnection

24.4 Video Testing

Test:

- Video startup
- Video pause
- Video playback
- Video seeking
- End-of-file
- Missing video
- Unsupported video
- Frame drops
- Timestamp mismatch

24.5 Observation Testing

A complete observation test should verify:

```
Select frame
    |
    v
Create observation
    |
    v
Calculate/display coordinate
    |
    v
Generate grid reference
    |
    v
Display map marker
    |
    v
Save observation
    |
    v
Mission log updated
    |
    v
Export succeeds
```

24.6 Offline Testing

This is one of the most important test categories.

The test procedure should be:

1. Start with Internet connectivity.
2. Start the application.
3. Load required local resources.
4. Disconnect the machine from all networks.
5. Restart the application.
6. Load an offline map.

7. Load recorded video.
8. Start telemetry simulator.
9. Create observations.
10. Save mission.
11. Export mission data.

The expected result is that all required functions continue to work.

24.7 Failure Testing

The application should also be tested under failure conditions.

Examples:

- GPS unavailable
- Telemetry stops
- Video stops
- Invalid coordinates
- Database failure
- Disk space shortage
- Corrupt video
- Corrupt mission file
- Unexpected application shutdown

The application should provide clear status information rather than silently displaying stale data.

25 Test Matrix

Module	Test	Expected Result
Map	Load local map	Map loads without Internet
Map	Zoom / pan	Navigation works

Telemetry	Simulator start	Platform appears
Telemetry	Platform movement	Marker updates correctly
Grid	Coordinate conversion	Correct reference generated
Video	Load recording	Video plays correctly
Sync	Video + telemetry	Correct time association
Observation	Create marker	Observation saved
Database	Save mission	Data persists locally
Export	Export mission	Valid export generated
Offline	Disconnect network	Core functions continue
Failure	Stop telemetry	UI reports telemetry loss
Replay	Replay mission	Recorded state reconstructed

26 Data Integrity and Auditability

Every important event should have:

- Timestamp
- Source
- Platform ID
- Mission ID
- Event type

The system should preserve the distinction between:

Observed

!=

Calculated

!=

Confirmed

This prevents the application from presenting an automatically calculated result as if it were independently verified.

27 Security and Offline Reliability

Because the application is intended to work with sensitive operational data, security should be considered from the beginning.

Recommended measures include:

- Local-only data storage
- No unnecessary external network calls
- Input validation
- Database integrity checks
- Application logs
- Role-based access where required
- Secure local configuration
- Controlled export functionality
- Backup and recovery procedure

The application should also clearly display its current connectivity state.

For example:

```
NETWORK: OFFLINE  
MAP: LOCAL  
DATABASE: READY  
VIDEO: PLAYING  
TELEMETRY: ACTIVE
```

28 Performance Requirements

The application should be designed for continuous operation.

Important performance areas include:

- Map rendering
- Video decoding
- Telemetry processing
- Database writes

- Event logging
- Coordinate calculations

Telemetry processing should not block video playback.

Video rendering should not block map interaction.

Database writes should preferably use buffered/asynchronous operations where appropriate.

29 Hardware Integration Checklist

Before integrating a specific platform, collect:

1. Platform communication interface
2. Telemetry protocol
3. Telemetry message definitions
4. Coordinate reference system
5. Video output/interface
6. Video transport
7. Frame timestamp availability
8. Camera metadata
9. Gimbal metadata
10. Altitude source
11. Position accuracy information
12. Sensor calibration information

Without these details, accurate sensor-to-map association may not be possible.

30 Important Engineering Constraint

The most difficult part of the project is not displaying a video or placing a marker on a map.

The difficult component is determining how accurately an image observation can be associated with a geographic position.

The system needs to establish the relationship:

Image Coordinate $\rightarrow$ Sensor Geometry $\rightarrow$ Platform Pose $\rightarrow$ Ground Model $\rightarrow$ Geographic Coordinate

Errors in any of these inputs can cause geographic error.

Therefore, georeferencing should be developed and tested as an independent software module rather than embedding the calculation directly into the UI.

31 Recommended MVP

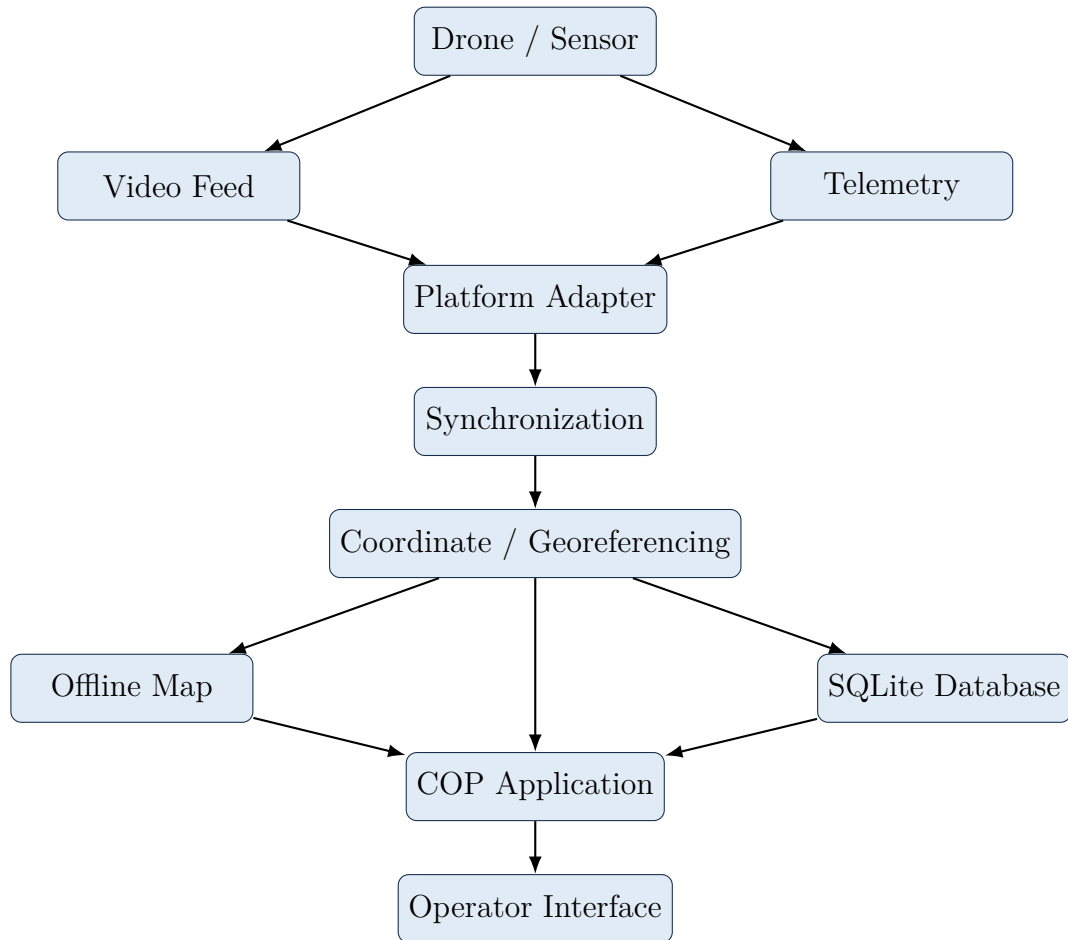
The first usable version should contain:

1. Desktop application
2. Offline map
3. Local map tiles
4. Coordinate display
5. Grid reference conversion
6. Telemetry simulator
7. Platform marker
8. Platform track
9. Recorded video playback
10. Video/telemetry synchronization
11. Observation creation
12. Geographic observation storage
13. Mission database
14. Mission log
15. Mission replay
16. CSV/JSON export
17. Offline test mode

Real hardware should be integrated after this baseline has passed testing.

32 Final Architecture

The complete conceptual system is:



The system can therefore be summarized as:

$$\text{Sensor Data} + \text{Telemetry} + \text{Time} + \text{Coordinate System} + \text{Offline Map} = \text{Geospatial COP}$$

(1)

33 Development Philosophy

The project should be developed from the inside out.

The recommended order is:

1. Data Model

|

2. Coordinate Engine

|

3. Telemetry Simulator
|
4. Offline Map
|
5. Video Playback
|
6. Synchronization
|
7. Observation / Georeferencing
|
8. Mission Database
|
9. Replay
|
10. Hardware Adapters

The major principle is:

Do not make the real drone the first dependency of the software.

The software should first be made functional using simulated telemetry and recorded sensor data.

Once the software architecture is stable, real hardware can replace the simulator through the platform adapter layer.

34 Conclusion

The proposed system is an offline Common Operating Picture application designed around modular sensor and telemetry integration.

The architecture separates:

- Hardware interfaces
- Telemetry
- Video
- Coordinate conversion
- Georeferencing
- Mapping

- Database
- User interface
- Mission replay
- Testing

This separation allows the project to begin immediately without waiting for a specific drone platform.

The development team can initially use:

- Simulated telemetry
- Recorded video
- Offline map data
- Known coordinate test points
- Synthetic missions

After the complete software pipeline works, platform-specific interfaces can be introduced.

The final system should therefore be viewed not as a “drone application” tied to one aircraft, but as a modular offline geospatial platform capable of receiving standardized sensor and telemetry information from different supported sources.